

تقرير مشروع المترجمات (Compilers Project Report)

عمل الطلاب :

محمد جودت محمود المعدني

زينب هيثم محمد

كوثر حميد الأحمد

عمر بسام تقوى

عبدالله تيسير حميد

الفهرس:

1.....	تقرير مشروع المترجمات (Compilers Project Report)
2.....	المقدمة (Introduction)
2.....	أولاً: python
2.....	1. التحليل اللغوي والنحوي (Lexical & Syntax Analysis)
2.....	1.1 المحلل اللفظي (Lexer)
2.....	1.2 المحلل القواعدي (Parser)
3.....	2. شجرة القواعد المجردة (Abstract Syntax Tree - AST)
3.....	2.1 الهيكلية العامة (Hierarchy)
3.....	2.2 تفصيل العقد (Full AST Nodes Description)
3.....	أولاً: عقد الجمل (Statement Nodes)
3.....	ثانياً: عقد التعبيرات (Expression Nodes)
4.....	3. بناء الشجرة (The Visitor Pattern)
4.....	4. جدول الرموز (Symbol Table)
4.....	4.1 هيكلية الجدول
4.....	4.2 آلية العمل (Scoping Rules)
5.....	ثانياً: HTML & Jinja2 & CSS
5.....	1. التحليل اللغوي والنحوي (Lexical & Syntax Analysis)
5.....	1.1 قواعد HTML و Jinja2 (HtmlJinjaParser)
5.....	1.2 قواعد CSS (CssParser)
5.....	2. شجرة القواعد المجردة (AST - Abstract Syntax Tree)
5.....	2.1 عقد HTML (ast.html)
6.....	2.2 عقد Jinja2 (ast.jinja)
6.....	2.3 عقد CSS (ast.css)
6.....	3. بناء الشجرة والتكامل (Visitor Pattern Integration)
6.....	4. التحليل الدلالي وجدول الرموز (Semantic Analysis & Symbol Table)
6.....	4.1 إدارة المجالات (Scope Management)
7.....	4.2 تعريف وحل الرموز
7.....	الخاتمة:

المقدمة (Introduction)

يهدف هذا المشروع إلى بناء مترجم (Compiler) يحاكي بيئة تطوير الويب باستخدام إطار العمل Flask ولغة Python، بالإضافة إلى محرك القوالب Jinja2 و HTML و CSS. يقوم النظام بتحليل الكود المصدري، والتحقق من صحة التراكيب اللغوية (Syntax)، وبناء هيكلية بيانات داخلية قوية (AST) تعتمد على مفاهيم البرمجة كائنية التوجه (OOP)، بالإضافة إلى إدارة الذاكرة والمجالات (Scopes) عبر جدول رموز متقدم (Symbol Table).

أولاً: python :

1. التحليل اللغوي والنحوي (Lexical & Syntax Analysis)

تم استخدام أداة ANTLR4 لتعريف قواعد اللغة، حيث تم تقسيم العمل إلى مرحلتين:

1.1 المحلل اللفظي (Lexer)

تم تعريف الرموز (Tokens) الأساسية للغة بايثون، وشملت:

الكلمات

المحجوزة: (def, class, if, else, while, return, import, try, except).

العمليات: الحسابية (+, -, *, /), المنطقية (and, or, not)، والمقارنات (>, ==, !=).

هيكلية اللغة: معالجة دقيقة للمسافات البادئة (INDENT, DEDENT) والأسطر الجديدة (NEWLINE) التي تميز بنية بايثون.

1.2 المحلل القواعدي (Parser)

تم بناء ملف قواعد (PythonParser.g4) يغطي الهياكل التالية:

الجمل البسيطة (Simple Statements): الإسناد، استدعاء الدوال، التعبيرات الحسابية.

الجمل المركبة (Compound Statements): الدوال، الكلاسات، الحلقات، والجمل الشرطية.

قواعد Flask: دعم هيكلية الـ Decorators (مثل @app.route) كجزء أساسي من تعريف الدوال.

2. شجرة القواعد المجردة (Abstract Syntax Tree - AST)

تعتبر الـ AST المكون الأهم في المشروع، حيث تم تصميمها كشجرة كائنات (Object Tree) باستخدام لغة Java ومفاهيم الوراثة (Inheritance).

2.1 الهيكلية العامة (Hierarchy)

ترث جميع العقد من الفئة الأساسية المجردة `ASTNode`، والتي تحتوي على الخصائص المشتركة:

- `lineNumber`: لتتبع موقع الكود.
- `nodeName`: اسم نوع العقدة.
- `printTree()`: دالة مساعدة لطباعة الشجرة بشكل هرمي مقروء.

2.2 تفصيل العقد (Full AST Nodes Description)

تنقسم العقد إلى نوعين رئيسيين:

أولاً: عقد الجمل (Statement Nodes)

وهي العقد التي تمثل أوامر تنفيذية ولا تعيد قيمة مباشرة:

1. `BlockNode`: يمثل كتلة برمجية تحتوي على قائمة من الجمل (مثل جسم الدالة أو محتوى `if`).
2. `FunctionDefNode`: لتعريف الدالة، تحتوي على: الاسم، قائمة البارامترات، الجسم (`BlockNode`)، وقائمة الـ `Decorators`.
3. `ClassDefNode`: لتعريف الكلاس، تحتوي على الاسم والجسم.
4. `AssignmentNode`: تمثل عملية الإسناد (`target = value`).
5. `AugmentedAssignNode`: للإسناد المدمج بعملية (مثل `x += 1`).
6. `IfNode`: تحتوي على الشرط، جسم `then`، وجسم `else`.
7. `WhileNode`: تمثل حلقة التكرار المشروطة.
8. `ForNode`: تمثل حلقة التكرار على العناصر (`for i in list`).
9. `ReturnNode`: جملة الإرجاع من الدالة.
10. `ImportNode` / `FromImportNode`: لتمثيل جمل الاستيراد المختلفة.
11. `TryNode` / `ExceptionHandlerNode`: لمعالجة الاستثناءات والأخطاء.
12. `FlowControlNode`: للجمل البسيطة (مثل `break`, `continue`, `pass`).

ثانياً: عقد التعبيرات (Expression Nodes)

وهي العقد التي يتم تقييمها لإنتاج قيمة:

1. `BinaryOpNode`: للعمليات الثنائية (`left operator right`).
2. `UnaryOpNode`: للعمليات الأحادية (`not x`, `-5`).
3. `TernaryOpNode`: للجملة الشرطية السطرية (`x if cond else y`).
4. `LiteralNode`: للقيم الثابتة (الأعداد الصحيحة، النصوص، القيم المنطقية).
5. `IdentifierNode`: تمثل أسماء المتغيرات.

6. `FunctionCallNode`: استدعاء دالة مع قائمة الوسائط (Arguments).
 7. `ListNode`: تمثل القوائم `[1, 2, 3]`.
 8. `DictNode`: تمثل القواميس `{key: value}`.
 9. `SubscriptNode`: للوصول لعناصر المصفوفات `arr[i]`.
 10. `AttributeAccessNode`: للوصول لخصائص الكائنات `obj.field`.
-

3. بناء الشجرة (The Visitor Pattern)

لتحويل شجرة الإعراب (Parse Tree) الخاصة بـ ANTLR إلى الـ AST الخاصة بنا، تم بناء كلاس `ASTBuilder`.

- **الوراثة:** يرث من `<PythonParserBaseVisitor<ASTNode`.
 - **الوظيفة:** يزور كل سياق (Context) ويستخرج المعلومات اللازمة (الأسماء، الأسطر، الأبناء) وينشئ كائن الـ AST المقابل.
 - **التعامل الذكي:** يعالج الحالات المعقدة مثل تحويل الـ `elif` المتعددة إلى جمل `if` متداخلة (Nested ifs) لتبسيط الشجرة.
-

4. جدول الرموز (Symbol Table)

تم تصميم نظام إدارة الذاكرة والمجالات لمحاكاة سلوك بايثون (Dynamic Scoping Concept).

4.1 هيكلية الجدول

1. `Symbol`: الكلاس الأساسي لكل معرف (متغير، دالة).
2. `Scope (Interface)`: واجهة تحدد وظائف المجال (`define, resolve`).
3. `FunctionSymbol`: يعتبر "رمزاً" في المجال الأب، ويعتبر "مجالاً" (`Scope`) بحد ذاته يحتوي على البارامترات والمتغيرات المحلية.
4. `GlobalScope`: المجال العام الذي يحتوي على الدوال الرئيسية والمتغيرات العامة.

4.2 آلية العمل (Scoping Rules)

- عند مرور الـ Visitor بتعريف دالة، يتم إنشاء `Scope` جديد.
 - يتم ربط المجال الجديد بالمجال المحيط (`Enclosing Scope`).
 - يتم تخزين المتغيرات والبارامترات داخل المجال النشط حالياً.
 - عند البحث عن متغير، يتم البحث تصاعدياً من المجال الحالي وصولاً إلى الـ `Global`.
-

ثانياً: HTML & Jinja2 & CSS :

نظراً لطبيعة تطوير الويب المتداخلة، تم تصميم المترجم ليعالج لغة HTML المدمجة مع منطق Jinja2، بالإضافة إلى القدرة على تحليل تنسيقات CSS سواء كانت ضمن ملفات مستقلة أو مدمجة داخل وسوم `<style>`.

1. التحليل اللغوي والنحوي (Lexical & Syntax Analysis)

تم استخدام ANTLR4 لبناء ملفات القواعد، حيث تم تقسيم العمل إلى جزأين رئيسيين:

1.1 قواعد HTML و Jinja2 (HtmlJinjaParser)

تم دمج قواعد اللغتين في ملف واحد (`HtmlJinjaParser.g4`) للسماح بالتداخل الطبيعي بينهما.

● **المحلل اللفظي (Lexer):** يتعرف على وسوم (`HTML (TAG_OPEN, TAG_NAME)`) والنصوص الصريحة، بالإضافة إلى رموز Jinja المميزة مثل `{ { ... } }` للمتغيرات و `{ % ... }` للجمل التحكمية.

● **المحلل القواعدي (Parser):** يعالج الهيكلية الشجرية لـ HTML (عناصر متداخلة) ويدمج معها جمل التحكم الخاصة بـ Jinja (مثل `if, for`) كعقد حاوية (Container Nodes) تحيط بمحتوى HTML.

1.2 قواعد CSS (CssParser)

تم بناء محلل خاص لـ CSS (`cssParser.g4`) للتعامل مع تعقيدات التنسيق.

● **الرموز (Tokens):** تشمل المحددات (Selectors)، الخصائص (Properties)، القيم (Values)، والوحدات (`px, %, em`).

● **القواعد (Rules):** تغطي تعريف القواعد الأساسية (`Rulesets`)، وقواعد الوسائط (`media@`)، والرسوم المتحركة (`keyframes@`)، والمتغيرات (`var ( ... )`).

2. شجرة القواعد المجردة (AST - Abstract Syntax Tree)

تم توسيع هيكلية الـ AST لتشمل حزمًا (Packages) مخصصة لكل لغة (`ast.html, ast.jinja, ast.css`)، وجميعها ترث من الفئة الأم `Node`.

2.1 عقد HTML (ast.html)

تمثل البنية الهيكلية لصفحة الويب:

● **HtmlDocument:** جذر المستند الذي يحتوي على قائمة من العقد الأبناء.

● **HtmlElement:** يمثل وسم HTML (مثل `div, span`) ويحتوي على الاسم، السمات (Attributes)، وقائمة الأبناء.

● **HtmlAttribute:** يمثل الزوج (`"name="value"`) داخل الوسم.

● **HtmlChardata:** يمثل النصوص العادية الظاهرة في الصفحة.

● **Script / Style:** عقد خاصة للتعامل مع كود JavaScript أو CSS المضمن.

2.2 عقد Jinja2 (ast.jinja)

تمثل المنطق البرمجي الديناميكي داخل القالب:

● **JinjaExpression**: للعقد التي تطبع قيماً (`{{ expression }}`).

● **IfStatement / Elif / Else**: لتمثيل التفرع الشرطي، حيث تحتوي كل عقدة على شرط

(Condition) وجسم (Body).

● **ForStatement**: لتمثيل الحلقات التكرارية، تخزن متغير الحلقة والمصفوفة المراد تكرارها.

● **AssignmentStatement**: لتعريف المتغيرات داخل القالب (`{% set x = 1 %}`).

● **ExtendsStatement**: لدعم وراثة القوالب.

2.3 عقد CSS (ast.css)

تمثل قواعد التصميم:

● **CssStylesheet**: الجذر لملف الـ CSS.

● **CssRuleSet**: تربط بين المحددات (selectors) وقائمة التصريحات

(declarations).

● **CssDeclaration**: تمثل الخاصية والقيمة (مثل `color: red`).

● **CssMediaRule**: تمثل قواعد التصميم المتجاوب (Media Queries) وتحتوي على قواعد

فرعية.

● **CssKeyframesRule**: لتمثيل خطوات الرسوم المتحركة.

3. بناء الشجرة والتكامل (Visitor Pattern Integration)

تعتبر عملية بناء الـ AST هنا عملية "هجينة" وذكية، حيث يتم دمج المحللات المختلفة معاً عبر

الكلاسات `CssVisitor` و `HtmlJinjaVisitor`.

● **التكامل (Embedding)**: عند مرور `HtmlJinjaVisitor` بوسم `<style>` أو

سمة `=style "..."`، فإنه لا يعامل المحتوى كنص عادي، بل يقوم

باستدعاء `CssParser` وتحويل النص إلى شجرة **CSS AST** فرعية، ثم يربطها بشجرة الـ HTML

الرئيسية.

● **المعالجة الديناميكية**: يقوم الـ Visitor بتحويل هياكل `{% if %}` و `for %}`

`{%}` المسطحة في الـ Parse Tree إلى هيكلية هرمية متداخلة في الـ AST لسهولة المعالجة

لاحقاً.

4. التحليل الدلالي وجدول الرموز (Semantic Analysis & Symbol Table)

تم تنفيذ `SymbolTableVisitor` الذي يمر على شجرة الـ AST الموحدة لإدارة المجالات (Scopes)

والتحقق من المعارف.

4.1 إدارة المجالات (Scope Management)

يعتمد النظام على مكدرس مجالات (Scope Stack) في `SymbolTableManager` لمحاكاة التداخل:

● **Global Scope**: يحتوي على المتغيرات العامة.

● **Local Scopes**: يتم إنشاء مجال جديد تلقائياً عند الدخول في:

● حلقات التكرار (`ForStatement`) في Jinja.

● الجمل الشرطية (IfStatement).

● وسوم <style> وقواعد @media في CSS (لعزل المتغيرات أو التعريفات

الخاصة).

4.2 تعريف وحل الرموز

● **Jinja**: يتم تسجيل متغيرات الحلقة (`for i in items`) ومتغيرات الإسناد (`= set x`) ... في الجدول للتحقق من استخدامها لاحقاً.
● **CSS**: يتم التعرف على متغيرات CSS (`--variable-name`) وميزات الخطوط (`font-` `feature-values`) وتسجيلها في الجدول لضمان صحة المراجع (`var(...)`).

الخاتمة :

بهذا يكتمل هيكل المترجم ليغطي الجانب الخلفي (Python/Flask) والواجهة الأمامية (HTML/CSS/Jinja2). يتميز النظام بقدرته على:

1. تحليل لغات متعددة ومتداخلة في آن واحد.
2. بناء شجرة AST موحدة وشاملة تعكس الهيكلية المنطقية للكود.
3. إدارة الذاكرة والتحقق من النطاقات (Scopes) عبر اللغات المختلفة، مما يمهد الطريق لمراحل التلخيص (Code Generation) أو التحسين (Optimization).